

CODE REPORT

BlazeDemo Automation Framework

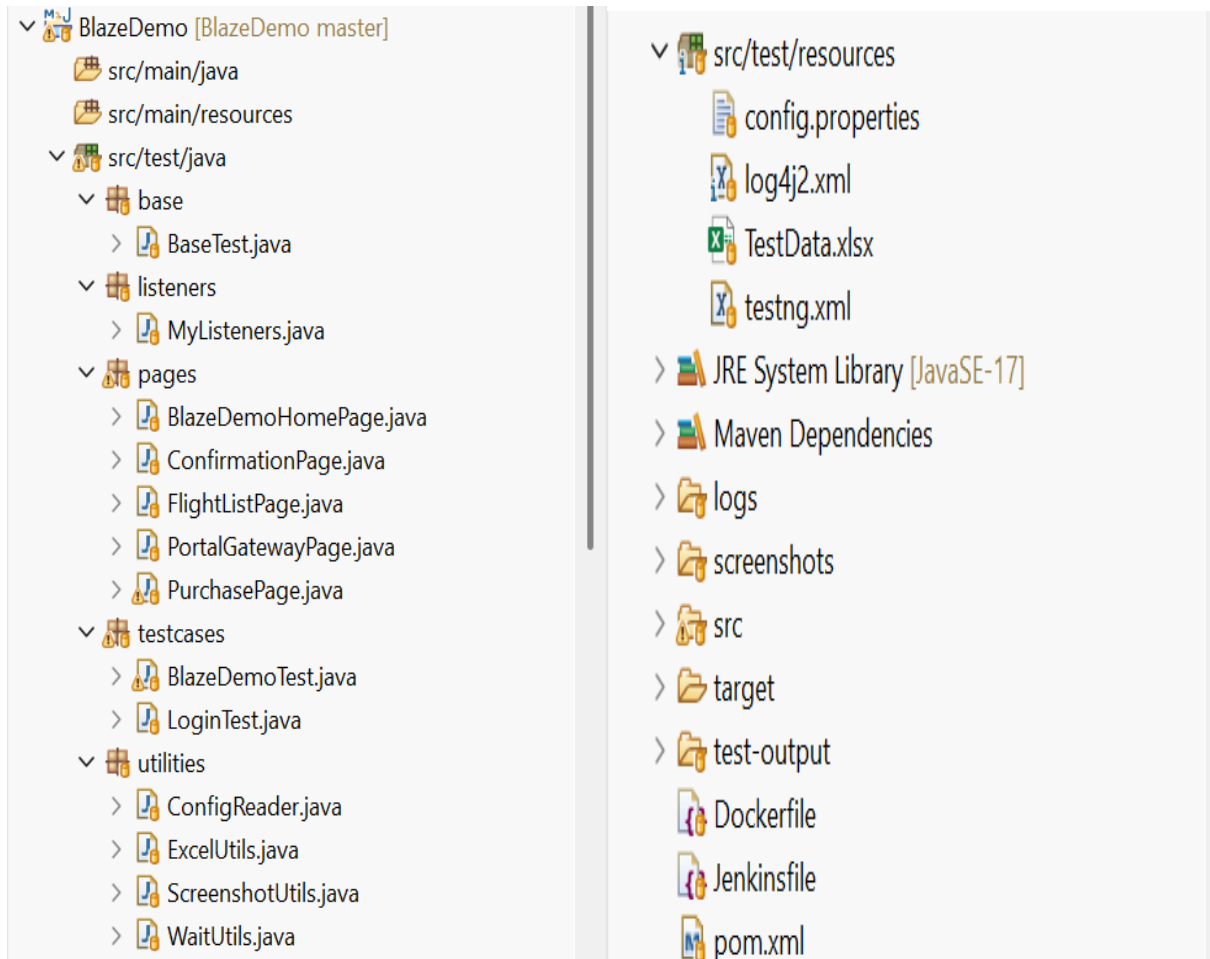
Project Name: BlazeDemo Flight Booking Automation Framework

Prepared By: Ayush Sharma

Github: https://github.com/AYUSH-YL/Capstone_project

Site: <https://blazedemo.com/reserve.php>

Project Structure:



CODE REPORT

CODE:

BaseTest.java:

```
package base;
```

```
import org.apache.logging.log4j.LogManager;
```

```
import org.apache.logging.log4j.Logger;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.chrome.ChromeDriver;
```

```
import org.openqa.selenium.chrome.ChromeOptions;
```

```
import org.testng.annotations.AfterClass;
```

```
import org.testng.annotations.BeforeClass;
```

```
import utilities.ScreenshotUtils;
```

```
public class BaseTest {
```

```
    protected static final Logger log = LogManager.getLogger(BaseTest.class);
```

```
    protected WebDriver driver;
```

```
    @BeforeClass(alwaysRun = true)
```

```
    public void setUp() {
```

```
        log.info("Initializing browser automation setup.");
```

```
        ChromeOptions options = new ChromeOptions();
```

```
        options.addArguments("--remote-allow-origins=*");
```

```
        options.addArguments("--no-sandbox");
```

```
        options.addArguments("--disable-dev-shm-usage");
```

CODE REPORT

```
if (System.getenv("RUNNING_IN_DOCKER") != null) {  
    log.info("System environment variable detected: Docker Pipeline. Forcing headless  
configuration flags.");  
    options.addArguments("--headless=new");  
    options.addArguments("--disable-gpu");  
    options.addArguments("--window-size=1920,1080");  
    options.setBinary("/usr/bin/google-chrome");  
} else {  
    log.info("System environment variable missing: Local Eclipse Sandbox. Launching  
visible GUI browser panel.");  
}  
  
driver = new ChromeDriver(options);  
driver.manage().window().maximize();  
  
log.info("Chrome Browser instance initialized successfully.");  
}  
  
@AfterClass(alwaysRun = true)  
public void tearDown() {  
    if (driver != null) {  
        driver.quit();  
        log.info("Browser instance terminated cleanly.");  
    }  
}  
  
public WebDriver getDriver() {  
    return this.driver;  
}
```

CODE REPORT

```
public void takeCheckpointScreenshot(String stepName) {  
    log.info("Triggering manual checkpoint screenshot: " + stepName);  
    ScreenshotUtils.captureScreenshot(driver, stepName);  
}  
}
```

MyListeners.java:
package listeners;

```
import com.aventstack.extentreports.ExtentReports;  
import com.aventstack.extentreports.ExtentTest;  
import com.aventstack.extentreports.Status;  
import com.aventstack.extentreports.reporter.ExtentSparkReporter;  
import com.aventstack.extentreports.reporter.configuration.Theme;  
import org.testng.ITestContext;  
import org.testng.ITestListener;  
import org.testng.ITestResult;  
import base.BaseTest;  
  
public class MyListeners implements ITestListener {  
  
    private static ExtentReports extent;  
    private static ThreadLocal<ExtentTest> testThread = new ThreadLocal<>();  
  
    @Override  
    public void onStart(ITestContext context) {
```

CODE REPORT

```
ExtentSparkReporter sparkReporter = new
ExtentSparkReporter("target/ExtentReport.html");

    sparkReporter.config().setDocumentTitle("BlazeDemo Automation Report");
    sparkReporter.config().setReportName("End To End Regression Results");
    sparkReporter.config().setTheme(Theme.DARK);


    extent = new ExtentReports();
    extent.attachReporter(sparkReporter);
    extent.setSystemInfo("Environment", "QA Pipeline");
    extent.setSystemInfo("Author", "Ayush");
    extent.setSystemInfo("OS", System.getProperty("os.name"));
}


@Override
public void onTestStart(ITestResult result) {
    System.out.println("Starting Test Execution: " + result.getName());
    ExtentTest test = extent.createTest(result.getName());
    testThread.set(test);
}


@Override
public void onTestSuccess(ITestResult result) {
    System.out.println("TEST PASSED: " + result.getName());
    testThread.get().log(Status.PASS, "Test Case Passed: " + result.getName());
}


@Override
```

CODE REPORT

```
public void onTestFailure(ITestResult result) {

    System.out.println("TEST FAILED: " + result.getName() + " -> Taking automated
screenshot.");

    testThread.get().log(Status.FAIL, "Test Case Failed: " + result.getName());

    testThread.get().log(Status.FAIL, result.getThrowable());

    try {

        Object testClass = result.getInstance();

        org.openqa.selenium.WebDriver driver = ((BaseTest) testClass).getDriver();

        if (driver != null) {

            String screenshotPath = utilities.ScreenshotUtils.captureScreenshot(driver,
result.getName());

            testThread.get().addScreenCaptureFromPath("../" + screenshotPath);

        }

    } catch (Exception e) {

        System.out.println("Exception thrown while attaching screenshot to Extent Report: " +
e.getMessage());

    }

}

@Override

public void onTestSkipped(ITestResult result) {

    System.out.println("Test Skipped: " + result.getName());

    testThread.get().log(Status.SKIP, "Test Case Skipped: " + result.getName());

}

@Override
```

CODE REPORT

```
public void onFinish(ITestContext context) {  
    if (extent != null) {  
        extent.flush();  
    }  
}  
  
public static ExtentTest getTestLog() {  
    return testThread.get();  
}  
}
```

BlazeDemoHomePage.java:

```
package pages;
```

```
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;  
import org.openqa.selenium.support.ui.Select;  
import utilities.WaitUtils;
```

```
public class BlazeDemoHomePage {  
    private WebDriver driver;  
  
    private By departureDropdown = By.name("fromPort");  
    private By destinationDropdown = By.name("toPort");  
    private By findFlightsButton = By.cssSelector("input[type='submit'].btn-primary");  
    private By homeButtonLink = By.linkText("home");  
  
    public BlazeDemoHomePage(WebDriver driver) {
```

CODE REPORT

```
this.driver = driver;
}

public void clickHomeButtonLink() {
    WaitUtils.waitForElementClickable(driver, homeButtonLink, 10).click();
}

public void selectDepartureCity(String departure) {
    WaitUtils.waitForElementVisible(driver, departureDropdown, 10);
    Select select = new Select(driver.findElement(departureDropdown));
    select.selectByValue(departure);
}

public void selectDestinationCity(String destination) {
    Select select = new Select(driver.findElement(destinationDropdown));
    select.selectByValue(destination);
}

public void clickFindFlights() {
    driver.findElement(findFlightsButton).click();
}
}
```

ConfirmationPage.java:

```
package pages;
```

```
import org.openqa.selenium.By;
```


CODE REPORT

```
import org.openqa.selenium.WebDriver;
import utilities.WaitUtils;

public class ConfirmationPage {
    private WebDriver driver;

    private By thankYouHeader = By.xpath("//h1[contains(text(),'Thank you for your purchase today!')]");

    public ConfirmationPage(WebDriver driver) {
        this.driver = driver;
    }

    public String getConfirmationMessage() {
        return WaitUtils.waitForElementVisible(driver, thankYouHeader, 10).getText().trim();
    }
}
```

FlightList.java:
package pages;

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import utilities.WaitUtils;

public class FlightListPage {
    private WebDriver driver;
```

CODE REPORT

```
private By flightsHeader = By.xpath("//h3[contains(text(),'Flights from')]");

private By chooseFlightButton = By.cssSelector("input[type='submit'][value='Choose This Flight']");


public FlightListPage(WebDriver driver) {
    this.driver = driver;
}

public boolean isFlightListDisplayed() {
    return WaitUtils.waitForElementVisible(driver, flightsHeader, 10).isDisplayed();
}

public void chooseFirstAvailableFlight() {
    WaitUtils.waitForElementClickable(driver, chooseFlightButton, 10).click();
}
}
```

PortalGatewayPage.java:
package pages;

```
import org.openqa.selenium.By;
import org.openqa.selenium.WebDriver;
import utilities.WaitUtils;

public class PortalGatewayPage {
    private WebDriver driver;

    private By loginEmailField = By.id("email");
```

CODE REPORT

```
private By loginPasswordField = By.id("password");
```

```
private By loginSubmitButton = By.cssSelector("button[type='submit']");
```

```
private By registerLinkButton = By.linkText("Register");
```

```
private By regNameField = By.id("name");
```

```
private By regCompanyField = By.id("company");
```

```
private By regEmailField = By.id("email");
```

```
private By regPasswordField = By.id("password");
```

```
private By regConfirmPasswordField = By.id("password-confirm");
```

```
private By regSubmitButton = By.cssSelector("button[type='submit']");
```

```
public PortalGatewayPage(WebDriver driver) {
```

```
    this.driver = driver;
```

```
}
```

```
public void clickRegisterLink() {
```

```
    WaitUtils.waitForElementClickable(driver, registerLinkButton, 10).click();
```

```
}
```

```
public void executeNegativeRegistration(String name, String company, String email, String password) {
```

```
    WaitUtils.waitForElementVisible(driver, regNameField, 10);
```

```
    driver.findElement(regNameField).sendKeys(name);
```

```
    driver.findElement(regCompanyField).sendKeys(company);
```

```
    driver.findElement(regEmailField).sendKeys(email);
```

```
    driver.findElement(regPasswordField).sendKeys(password);
```

CODE REPORT

```
driver.findElement(regConfirmPasswordField).sendKeys(password);  
driver.findElement(regSubmitButton).click();  
}
```

```
public void executeNegativeLogin(String email, String password) {  
    WaitUtils.waitForElementVisible(driver, loginEmailField, 10);  
    driver.findElement(loginEmailField).sendKeys(email);  
    driver.findElement(loginPasswordField).sendKeys(password);  
    driver.findElement(loginSubmitButton).click();  
}
```

```
public boolean verify419ErrorPageState() {  
    String source = driver.getPageSource();  
    return source.contains("419") || source.contains("Page Expired");  
}
```

```
public void navigateBackTwoSteps() {  
    driver.navigate().back();  
    driver.navigate().back();  
}  
}
```

PurchasePage.java:
package pages;

```
import org.openqa.selenium.By;  
import org.openqa.selenium.WebDriver;
```

CODE REPORT

```
import org.openqa.selenium.support.ui.Select;
import utilities.WaitUtils;
import org.openqa.selenium.support.ui.Select;
import org.openqa.selenium.NoSuchElementException;
import org.apache.logging.log4j.LogManager;
import org.apache.logging.log4j.Logger;

public class PurchasePage {
    private static final Logger log = LogManager.getLogger(PurchasePage.class);
    private WebDriver driver;

    private By purchaseHeader = By.xpath("//h2[contains(text(),'Your flight')]");
    private By totalCostText = By.xpath("//p[contains(text(),'Total Cost:')] /em");

    private By inputName = By.id("inputName");
    private By addressField = By.id("address");
    private By cityField = By.id("city");
    private By stateField = By.id("state");
    private By zipCodeField = By.id("zipCode");
    private By cardTypeDropdown = By.id("cardType");
    private By creditCardNumberField = By.id("creditCardNumber");
    private By nameOnCardField = By.id("nameOnCard");
    private By purchaseFlightButton = By.cssSelector("input[type='submit'][value='Purchase Flight']");

    public PurchasePage(WebDriver driver) {
        this.driver = driver;
    }
}
```

CODE REPORT

```
}
```

```
public boolean isPurchasePageDisplayed() {  
    return WaitUtils.waitForElementVisible(driver, purchaseHeader, 10).isDisplayed();  
}
```

```
public String getTotalCost() {  
    return driver.findElement(totalCostText).getText().trim();  
}
```

```
public void fillPersonalAndPaymentDetails(String name, String address, String city, String  
state, String zip, String cardType, String cardNumber, String cardName) {
```

```
    driver.findElement(inputName).sendKeys(name);  
    driver.findElement(addressField).sendKeys(address);  
    driver.findElement(cityField).sendKeys(city);  
    driver.findElement(stateField).sendKeys(state);  
    driver.findElement(zipCodeField).sendKeys(zip);
```

```
    Select selectCard = new Select(driver.findElement(cardTypeDropdown));
```

```
    try {  
        selectCard.selectByValue(cardType.toLowerCase());  
    } catch (NoSuchElementException e) {  
        log.warn("Card type '" + cardType + "' not found. Defaulting to 'Visa'.");  
        selectCard.selectByVisibleText("Visa");  
    }
```

```
    driver.findElement(creditCardNumberField).sendKeys(cardNumber);
```

CODE REPORT

```
        driver.findElement(nameOnCardField).sendKeys(cardName);
    }

    public void clickPurchaseFlight() {
        driver.findElement(purchaseFlightButton).click();
    }
}
```

BlazedemoTest.java:

```
package testcases;

import org.testng.Assert;
import org.testng.annotations.DataProvider;
import org.testng.annotations.Factory;
import org.testng.annotations.Test;

import base.BaseTest;
import pages.BlazeDemoHomePage;
import pages.ConfirmationPage;
import pages.FlightListPage;
import pages.PurchasePage;
import utilities.ExcelUtils;

public class BlazeDemoTest extends BaseTest {

    private BlazeDemoHomePage homePage;
    private FlightListPage flightListPage;
```

CODE REPORT

```
private PurchasePage purchasePage;
```

```
private ConfirmationPage confirmationPage;
```

```
private final String departure;
```

```
private final String destination;
```

```
private final String name;
```

```
private final String address;
```

```
private final String city;
```

```
private final String state;
```

```
private final String zip;
```

```
private final String cardType;
```

```
private final String cardNumber;
```

```
private final String cardName;
```

```
@DataProvider(name = "bookingData")
```

```
public static Object[][] bookingData() {
```

```
    return ExcelUtils.getTestData("src/test/resources/TestData.xlsx", "BookingSheet");
```

```
}
```

```
@Factory(dataProvider = "bookingData")
```

```
public BlazeDemoTest(String departure, String destination, String name, String address,
```

```
    String city, String state, String zip, String cardType,
```

```
    String cardNumber, String cardName) {
```

```
    this.departure = departure;
```

```
    this.destination = destination;
```

```
    this.name = name;
```

```
    this.address = address;
```


CODE REPORT

```
this.city = city;  
this.state = state;  
this.zip = zip;  
this.cardType = cardType;  
this.cardNumber = cardNumber;  
this.cardName = cardName;  
}
```

```
@Test(priority = 1, groups = { "end-to-end" })  
public void step2_searchFlights() {  
    log.info("Selecting Route -> " + departure + " to " + destination);  
  
    driver.get(utilities.ConfigReader.getUrl());  
  
    homePage = new BlazeDemoHomePage(driver);  
    Assert.assertEquals(driver.getTitle(), "BlazeDemo", "Page title mismatch on Home Page");  
    Assert.assertEquals(driver.getCurrentUrl(), "https://blazedemo.com/index.php", "Home  
Page URL mismatch");  
  
    homePage.selectDepartureCity(departure);  
    homePage.selectDestinationCity(destination);  
    homePage.clickFindFlights();  
}  
  
@Test(priority = 2, dependsOnMethods = { "step2_searchFlights" }, groups = { "end-to-end"  
})  
public void step3_validateAndSelectFlight() {  
    log.info("Verifying flight grid selection table.");
```

CODE REPORT

```
flightListPage = new FlightListPage(driver);
Assert.assertTrue(flightListPage.isFlightListDisplayed(),
    "Validation Failure: Flights listing table is missing.");
flightListPage.chooseFirstAvailableFlight();
}

@Test(priority = 3, dependsOnMethods = { "step3_validateAndSelectFlight" }, groups = {
"end-to-end" })

public void step4_fillPassengerFormAndBook() {
    log.info("Filling passenger details for: " + name);

    purchasePage = new PurchasePage(driver);
    Assert.assertTrue(purchasePage.isPurchasePageDisplayed(),
        "Validation Failure: Purchase page header not displayed.");

    String cleanName = name.replaceAll("[^a-zA-Z0-9]", "");

    purchasePage.fillPersonalAndPaymentDetails(name, address, city, state, zip,
        cardType, cardNumber, cardName);

    takeCheckpointScreenshot("FormInputCheckpoint_Passenger_" + cleanName);
    purchasePage.clickPurchaseFlight();
}

@Test(priority = 4, dependsOnMethods = { "step4_fillPassengerFormAndBook" }, groups = {
"end-to-end" })

public void step5_validateConfirmationReceipt() {
    log.info("Asserting completion receipts for passenger: " + name);
```

CODE REPORT

```
confirmationPage = new ConfirmationPage(driver);
String confirmationMessage = confirmationPage.getConfirmationMessage();

Assert.assertEquals(confirmationMessage, "Thank you for your purchase today!",
    "Validation Failure: Success banner mismatch!");

String cleanName = name.replaceAll("[^a-zA-Z0-9]", "");
takeCheckpointScreenshot("FinalBookingConfirmationReceipt_" + cleanName);
}
}
```

LoginTest.java:

```
package testcases;
```

```
import base.BaseTest;
```

```
import org.testng.Assert;
```

```
import org.testng.annotations.Test;
```

```
import pages.BlazeDemoHomePage;
```

```
import pages.PortalGatewayPage;
```

```
public class LoginTest extends BaseTest {
```

```
    private BlazeDemoHomePage homePage;
```

```
    private PortalGatewayPage portalPage;
```

```
    @Test(priority = 1, groups = { "end-to-end" })
```

CODE REPORT

```
public void validateBrokenPortalGateways() {  
    log.info("Starting Portal Security Regression Sweep (Once).");  
  
    String appUrl = utilities.ConfigReader.getUrl();  
    log.info("Navigating directly to application index: " + appUrl);  
    driver.get(appUrl);  
  
    homePage = new BlazeDemoHomePage(driver);  
    portalPage = new PortalGatewayPage(driver);  
  
    homePage.clickHomeButtonLink();  
    portalPage.clickRegisterLink();  
  
    log.info("Injecting hardcoded registration placeholders.");  
    portalPage.executeNegativeRegistration("DemoUser", "Wipro", "test@wipro.com",  
    "Password123");  
  
    takeCheckpointScreenshot("Registration_419_Error_Page");  
    Assert.assertTrue(portalPage.verify419ErrorPageState(), "Error: Registration failed to  
trigger a 419 Page state.");  
    log.info("Known error : Registration 419 Page.");  
  
    portalPage.navigateBackTwoSteps();  
  
    log.info("Injecting hardcoded login placeholders.");  
    portalPage.executeNegativeLogin("login@wipro.com", "SecurePass99");  
  
    takeCheckpointScreenshot("Login_419_Error_Page");
```

CODE REPORT

```
Assert.assertTrue(portalPage.verify419ErrorPageState(), "Error: Login failed to trigger a 419 Page state.");
```

```
log.info("Known error : 419 Page.");
```

```
portalPage.navigateBackTwoSteps();
```

```
log.info("Portal Sweep Complete.");
```

```
}
```

```
}
```

ConfigReader.java:

```
package utilities;
```

```
import java.io.FileInputStream;
```

```
import java.io.IOException;
```

```
import java.util.Properties;
```

```
public class ConfigReader {
```

```
    private static Properties properties;
```

```
    private static final String PROPERTY_FILE_PATH = "src/test/resources/config.properties";
```

```
    static {
```

```
        try (FileInputStream fileInputStream = new FileInputStream(PROPERTY_FILE_PATH)) {
```

```
            properties = new Properties();
```

```
            properties.load(fileInputStream);
```

```
        } catch (IOException e) {
```

```
            System.out.println("Critical Exception: Failed to load environment properties configuration mapping file at " + PROPERTY_FILE_PATH);
```

CODE REPORT

```
e.printStackTrace();

    throw new RuntimeException("Configuration file initialization aborted due to critical I/O
stream failure.");
}
}

public static String getProperty(String key) {
    String value = properties.getProperty(key);
    if (value == null) {
        System.out.println("Warning: Requested configuration mapping key '" + key + "' was not
found inside your properties structure.");
    }
    return value;
}

public static String getUrl() {
    return getProperty("url");
}

public static String getBrowser() {
    return getProperty("browser");
}

public static int getExplicitWaitTimeout() {
    return Integer.parseInt(getProperty("explicitWait"));
}
}
```

CODE REPORT

```
ExcelUtils.java:  
package utilities;
```

```
  
import java.io.FileInputStream;  
import java.io.IOException;  
import org.apache.poi.ss.usermodel.DataFormatter;  
import org.apache.poi.xssf.usermodel.XSSFCell;  
import org.apache.poi.xssf.usermodel.XSSFRow;  
import org.apache.poi.xssf.usermodel.XSSFSheet;  
import org.apache.poi.xssf.usermodel.XSSFWorkbook;
```

```
  
public class ExcelUtils {
```

```
  
    public static Object[][] getTestData(String filePath, String sheetName) {
```

```
        XSSFWorkbook workbook = null;
```

```
        FileInputStream fileInputStream = null;
```

```
        Object[][] data = null;
```

```
  
        try {
```

```
            fileInputStream = new FileInputStream(filePath);
```

```
            workbook = new XSSFWorkbook(fileInputStream);
```

```
            XSSFSheet sheet = workbook.getSheet(sheetName);
```

```
  
            int totalRows = sheet.getLastRowNum();
```

```
            XSSFRow headerRow = sheet.getRow(0);
```

```
            int totalColumns = headerRow.getLastCellNum();
```

CODE REPORT

```
data = new Object[totalRows][totalColumns];

DataFormatter formatter = new DataFormatter();

for (int i = 1; i <= totalRows; i++) {
    XSSFRow row = sheet.getRow(i);
    for (int j = 0; j < totalColumns; j++) {
        if (row == null) {
            data[i - 1][j] = "";
        } else {
            XSSFCell cell = row.getCell(j);
            data[i - 1][j] = formatter.formatCellValue(cell);
        }
    }
}

} catch (IOException e) {
    System.out.println("Critical Exception: Failed to read from Excel data matrix at path " +
filePath);
    e.printStackTrace();
} finally {
    try {
        if (workbook != null) workbook.close();
        if (fileInputStream != null) fileInputStream.close();
    } catch (IOException e) {
        System.out.println("Exception encountered while closing Excel streams: " +
e.getMessage());
    }
}

return data;
}
```


CODE REPORT

```
}
```

ScreenshotUtils.java:

```
package utilities;
```

```
import java.io.File;
```

```
import java.io.IOException;
```

```
import java.text.SimpleDateFormat;
```

```
import java.util.Date;
```

```
import org.openqa.selenium.OutputType;
```

```
import org.openqa.selenium.TakesScreenshot;
```

```
import org.openqa.selenium.WebDriver;
```

```
import org.openqa.selenium.io.FileHandler;
```

```
public class ScreenshotUtils {
```

```
    public static String captureScreenshot(WebDriver driver, String screenshotName) {
```

```
        String timestamp = new SimpleDateFormat("yyyyMMdd_HH:mm:ss").format(new Date());
```

```
        String fileName = screenshotName + "_" + timestamp + ".png";
```

```
        File directory = new File("screenshots");
```

```
        if (!directory.exists()) {
```

```
            directory.mkdir();
```

```
        }
```

```
        File source = ((TakesScreenshot) driver).getScreenshotAs(OutputType.FILE);
```

```
        String destinationPath = "screenshots/" + fileName;
```

CODE REPORT

```
File destination = new File(destinationPath);

try {

    FileHandler.copy(source, destination);

    System.out.println("Screenshot captured successfully: " + destinationPath);

} catch (IOException e) {

    System.out.println("Failed to capture screenshot: " + e.getMessage());

}

return destinationPath;

}

}
```

WaitUtils.java:
package utilities;

```
import java.time.Duration;

import org.openqa.selenium.By;

import org.openqa.selenium.WebDriver;

import org.openqa.selenium.WebElement;

import org.openqa.selenium.support.ui.ExpectedConditions;

import org.openqa.selenium.support.ui.WebDriverWait;

public class WaitUtils {

    public static WebElement waitForElementVisible(WebDriver driver, By locator, int
timeoutInSeconds) {

        WebDriverWait wait = new WebDriverWait(driver,
Duration.ofSeconds(timeoutInSeconds));

        return wait.until(ExpectedConditions.visibilityOfElementLocated(locator));

    }

}
```

CODE REPORT

```
}
```

```
public static WebElement waitForElementClickable(WebDriver driver, By locator, int  
timeoutInSeconds) {
```

```
    WebDriverWait wait = new WebDriverWait(driver,  
Duration.ofSeconds(timeoutInSeconds));
```

```
    return wait.until(ExpectedConditions.elementToBeClickable(locator));
```

```
}
```

```
}
```

Config.properties:

url=https://blazedemo.com/index.php

browser=chrome

implicitWait=10

explicitWait=10

excelTestDataPath=src/test/resources/TestData.xlsx

log4j2.xml:

<?xml version="1.0" encoding="UTF-8"?>

<Configuration status="WARN">

<Properties>

<Property name="baseDir">logs</Property>

</Properties>

<Appenders>

CODE REPORT

```
<Console name="ConsoleAppender" target="SYSTEM_OUT">
    <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %logger{36} -
%msg%n"/>
</Console>

<RollingFile name="FileAppender" fileName="${baseDir}/automation.log"
    filePattern="${baseDir}/automation-%d{yyyy-MM-dd}-%i.log.gz">
    <PatternLayout pattern="%d{yyyy-MM-dd HH:mm:ss} [%t] %-5level %logger{36} -
%msg%n"/>
    <Policies>
        <SizeBasedTriggeringPolicy size="10 MB" />
    </Policies>
    <DefaultRolloverStrategy max="5"/>
</RollingFile>
</Appenders>

<Loggers>
    <Root level="INFO">
        <AppenderRef ref="ConsoleAppender"/>
        <AppenderRef ref="FileAppender"/>
    </Root>
</Loggers>
</Configuration>
```

TestData.xlsx:

CODE REPORT

departure	destination	name	address	city	state	zip	cardType	cardNumber	cardName
Paris	London	John##Doe!!	123 Tech Lane	Noida	UP	201301	Visa	FOUR_ONE_ON E_ONE	John Doe
Portland	Rome	Amit_Sharma_\$ \$	456 IT Hub	Bangalore	Karnataka	560001	Mastercard	XYZ-INVALID- CARD	Amit Sharma
Boston	Berlin	<Admin>	789 Web Drive	Mumbai	MH	400001	American Express	0000AAAA0000	Admin

Testng.xml:

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE suite SYSTEM "https://testng.org/testng-1.0.dtd">
```

```
<suite name="BlazeDemo Booking Suite">
```

```
  <listeners>
```

```
    <listener class-name="listeners.MyListeners" />
```

```
  </listeners>
```

```
  <test name="End To End Flight Booking Test">
```

```
    <groups>
```

```
      <run>
```

```
        <include name="end-to-end" />
```

```
      </run>
```

```
    </groups>
```

```
    <classes>
```

```
      <class name="testcases.LoginTest" />
```

```
      <class name="testcases.BlazeDemoTest" />
```

```
    </classes>
```

```
  </test>
```

```
</suite>
```

CODE REPORT

Pom.xml:

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
https://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>BlazeDemo</groupId>

  <artifactId>BlazeDemo</artifactId>

  <version>0.0.1-SNAPSHOT</version>


  <properties>

    <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>

  </properties>


  <dependencies>

    <dependency>

      <groupId>org.seleniumhq.selenium</groupId>

      <artifactId>selenium-java</artifactId>

      <version>4.20.0</version>

    </dependency>


    <dependency>

      <groupId>org.testng</groupId>

      <artifactId>testng</artifactId>

      <version>7.9.0</version>

    </dependency>


    <dependency>
```

CODE REPORT

```
<groupId>org.apache.poi</groupId>  
<artifactId>poi-ooxml</artifactId>  
<version>5.2.5</version>  
</dependency>
```

```
<dependency>  
  <groupId>org.apache.logging.log4j</groupId>  
  <artifactId>log4j-api</artifactId>  
  <version>2.23.1</version>  
</dependency>
```

```
<dependency>  
  <groupId>org.apache.logging.log4j</groupId>  
  <artifactId>log4j-core</artifactId>  
  <version>2.23.1</version>  
</dependency>
```

```
<dependency>  
  <groupId>com.aventstack</groupId>  
  <artifactId>extentreports</artifactId>  
  <version>5.1.1</version>  
</dependency>  
</dependencies>
```

```
<build>  
  <plugins>  
    <plugin>
```

CODE REPORT

```
<groupId>org.apache.maven.plugins</groupId>
<artifactId>maven-compiler-plugin</artifactId>
<version>3.13.0</version>
<configuration>
  <source>17</source>
  <target>17</target>
</configuration>
</plugin>

<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-surefire-plugin</artifactId>
  <version>3.5.3</version>
  <configuration>
    <suiteXmlFiles>
      <suiteXmlFile>src/test/resources/testng.xml</suiteXmlFile>
    </suiteXmlFiles>
  </configuration>
</plugin>
</plugins>
</build>
</project>
```

```
Jenkinsfile:
pipeline {
  agent any
```


CODE REPORT

```
stages {
  stage('Docker Environment Verification') {
    steps {
      echo 'Verifying that Jenkins has access to the local Docker engine.'
      bat 'docker --version'
    }
  }

  stage('Docker Build Lifecycle') {
    steps {
      echo 'Compiling image artifact layer from project Dockerfile specifications.'
      bat 'docker build -t testng-framework .'
    }
  }

  stage('Docker Container Regression Run') {
    steps {
      echo 'Spanning up sandboxed container node to launch TestNG framework headless
suite.'
      bat 'docker run --name blazedemo-run testng-framework'
    }
  }

  stage('Docker Container Regression Run') {
    steps {
      echo 'Clearing any stale containers from prior failed pipeline runs.'
      bat 'docker rm -f blazedemo-run || ver>nul'
```

CODE REPORT

```
    echo 'Spanning up sandboxed container node to launch TestNG framework headless
suite.'

    bat 'docker run --name blazedemo-run testng-framework'

    }

    }

}

post {
    always {
        echo 'Extracting isolated test results and screenshots from inside the closed container
instance.'

        bat 'docker cp blazedemo-run:/app/screenshots ./screenshots'

        bat 'docker cp blazedemo-run:/app/target/ExtentReport.html ./target/ExtentReport.html'

        bat 'docker cp blazedemo-run:/app/target/surefire-reports ./target/surefire-reports'

        echo 'Cleaning up execution workspace nodes.'

        bat 'docker rm blazedemo-run'

        junit '**/target/surefire-reports/*.xml'

        archiveArtifacts artifacts: 'screenshots/**/*', allowEmptyArchive: true

        archiveArtifacts artifacts: 'target/ExtentReport.html', allowEmptyArchive: true
    }
}

success {
    echo 'Success: Containerized pipeline execution passed completely!'
}

failure {
    echo 'Error: Regression failure detected during isolated execution.'
}
```

CODE REPORT

```
}  
}
```

Dockerfile:

```
FROM maven:3.8.8-eclipse-temurin-17
```

```
RUN apt-get update && apt-get install -y wget gnupg2 \
```

```
&& wget -q -O - https://dl-ssl.google.com/linux/linux_signing_key.pub | apt-key add - \
```

```
&& echo "deb [arch=amd64] http://dl.google.com/linux/chrome/deb/ stable main" >>  
/etc/apt/sources.list.d/google-chrome.list \
```

```
&& apt-get update && apt-get install -y google-chrome-stable \
```

```
&& rm -rf /var/lib/apt/lists/*
```

```
WORKDIR /app
```

```
COPY pom.xml .
```

```
RUN mvn dependency:go-offline -B
```

```
COPY src ./src
```

```
ENV RUNNING_IN_DOCKER=true
```

```
CMD ["mvn", "clean", "test"]
```

Code Execution Result:

CODE REPORT

Search:

Passed: 5Failed: 0Skipped: 0s: 1/1Methods: 5 (89763 ms)

All TestsFailed TestsSummary

BlazeDemo Booking Suite (5/0/0/0) (43.273 s)Failure Exception

End To End Flight Booking Test (43.273 s)

testcases.LoginTest

validateBrokenPortalGateways (24.029 s)

testcases.BlazeDemoTest

step2_searchFlights (3.874 s)

step3_validateAndSelectFlight (1.262 s)

step4_fillPassengerFormAndBook (2.2 s)

step5_validateConfirmationReceipt (11 s)

```
TEST PASSED: step5_validateConfirmationReceipt
2026-06-11 13:04:23 [main] INFO base.BaseTest - Browser instance terminated

=====
BlazeDemo Booking Suite
Total tests run: 5, Passes: 5, Failures: 0, Skips: 0
=====
```

End To End Regression ResultsJun 11, 2026 03:38:32 PM

Tests

validateBrokenPortalGateways

3:38:50 PM / 00:00:21:355

Pass

step2_searchFlights

3:39:26 PM / 00:00:04:249

Pass

step3_validateAndSelectFlight

3:39:30 PM / 00:00:01:827

Pass

step4_fillPassengerFormAndBook

3:39:32 PM / 00:00:03:435

Pass

step5_validateConfirmationReceipt

3:39:35 PM / 00:00:16:697

Pass

validateBrokenPortalGateways

06.11.2026 3:38:50 PM06.11.2026 3:39:11 PM00:00:21:355#test-id=1

STATUS	TIMESTAMP	DETAILS
Pass	3:39:11 PM	Test Case Passed: validateBrokenPortalGateways